

CONTENTS IN DETAIL

FOREWORD	xix
-----------------	------------

PREFACE	xxi
----------------	------------

ACKNOWLEDGMENTS	xxiii
------------------------	--------------

INTRODUCTION	xxv
---------------------	------------

Who Rust Is For	xxv
Teams of Developers	xxvi
Students	xxvi
Companies	xxvi
Open Source Developers	xxvi
People Who Value Speed and Stability	xxvi
Who This Book Is For	xxvii
How to Use This Book	xxvii
Resources and How to Contribute to This Book	xxix

1	
GETTING STARTED	1

Installation	1
Installing rustup on Linux or macOS	2
Installing rustup on Windows	3
Troubleshooting	3
Updating and Uninstalling	4
Reading the Local Documentation	4
Using Text Editors and IDEs	4
Hello, World!	4
Project Directory Setup	4
Rust Program Basics	5
Anatomy of a Rust Program	6
Compilation and Execution	6
Hello, Cargo!	7
Creating a Project with Cargo	8
Building and Running a Cargo Project	9
Building for Release	11
Leveraging Cargo's Conventions	11
Summary	12

2	
PROGRAMMING A GUESSING GAME	13

Setting Up a New Project	14
Processing a Guess	14
Storing Values with Variables	15
Receiving User Input	16

Handling Potential Failure with the Result Type	17
Printing Values with println! Placeholders	18
Testing the First Part	19
Generating a Secret Number	19
Increasing Functionality with a Crate	19
Generating a Random Number	22
Comparing the Guess to the Secret Number	23
Allowing Multiple Guesses with Looping	27
Quitting After a Correct Guess	28
Handling Invalid Input	28
Summary	30

3

COMMON PROGRAMMING CONCEPTS

31

Variables and Mutability	32
Declaring Constants	34
Shadowing	34
Data Types	36
Scalar Types	37
Compound Types	40
Functions	44
Parameters	45
Statements and Expressions	46
Functions with Return Values	48
Comments	49
Control Flow	50
if Expressions	50
Repetition with Loops	54
Summary	59

4

UNDERSTANDING OWNERSHIP

61

What Is Ownership?	61
Ownership Rules	63
Variable Scope	63
The String Type	64
Memory and Allocation	65
Ownership and Functions	71
Return Values and Scope	72
References and Borrowing	73
Mutable References	75
Dangling References	78
The Rules of References	80
The Slice Type	80
String Slices	82
Other Slices	86
Summary	86

5	USING STRUCTS TO STRUCTURE RELATED DATA	87
Defining and Instantiating Structs		88
Using the Field Init Shorthand		89
Creating Instances with Struct Update Syntax		90
Creating Different Types with Tuple Structs		91
Defining Unit-Like Structs		92
An Example Program Using Structs		93
Refactoring with Tuples		94
Refactoring with Structs		95
Adding Functionality with Derived Traits		96
Methods		99
Method Syntax		99
Methods with More Parameters		102
Associated Functions		103
Multiple impl Blocks		104
Summary		104
 6	 ENUMS AND PATTERN MATCHING	 105
Defining an Enum		106
Enum Values		106
The Option Enum		110
The match Control Flow Construct		112
Patterns That Bind to Values		114
The Option<T> Match Pattern		115
Matches Are Exhaustive		116
Catch-All Patterns and the _ Placeholder		117
Concise Control Flow with if let		118
Staying on the “Happy Path” with let...else		119
Summary		121
 7	 PACKAGES, CRATES, AND MODULES	 123
Packages and Crates		124
Control Scope and Privacy with Modules		127
Paths for Referring to an Item in the Module Tree		129
Exposing Paths with the pub Keyword		131
Starting Relative Paths with super		134
Making Structs and Enums Public		134
Bringing Paths into Scope with the use Keyword		136
Creating Idiomatic use Paths		138
Providing New Names with the as Keyword		139
Re-exporting Names with pub use		140
Using External Packages		140
Using Nested Paths to Clean Up use Lists		141
Importing Items with the Glob Operator		142
Separating Modules into Different Files		143
Summary		145

8	COMMON COLLECTIONS	147
Storing Lists of Values with Vectors		148
Creating a New Vector		148
Updating a Vector		149
Reading Elements of Vectors		149
Iterating Over the Values in a Vector		151
Using an Enum to Store Multiple Types		152
Dropping a Vector Drops Its Elements		153
Storing UTF-8 Encoded Text with Strings		153
Defining Strings		153
Creating a New String		154
Updating a String		155
Indexing into Strings		157
Slicing Strings		159
Iterating Over Strings		159
Handling the Complexities of Strings		160
Storing Keys with Associated Values in Hash Maps		160
Creating a New Hash Map		161
Accessing Values in a Hash Map		161
Managing Ownership in Hash Maps		162
Updating a Hash Map		163
Hashing Functions		165
Summary		165
9	ERROR HANDLING	167
Unrecoverable Errors with the panic! Macro		168
Recoverable Errors with Result		171
Matching on Different Errors		173
Propagating Errors		175
To panic! or Not to panic!		181
Examples, Prototype Code, and Tests		182
When You Have More Information Than the Compiler		182
Guidelines for Error Handling		183
Custom Types for Validation		184
Summary		186
10	GENERIC TYPES, TRAITS, AND LIFETIMES	187
Removing Duplication by Extracting a Function		188
Generic Data Types		190
In Function Definitions		191
In Struct Definitions		193
In Enum Definitions		195
In Method Definitions		195
Performance of Code Using Generics		197
Defining Shared Behavior with Traits		198
Defining a Trait		199
Implementing a Trait on a Type		199

Using Default Implementations	201
Using Traits as Parameters	203
Returning Types That Implement Traits	205
Using Trait Bounds to Conditionally Implement Methods.	206
Validating References with Lifetimes	207
Dangling References	208
The Borrow Checker	209
Generic Lifetimes in Functions	210
Lifetime Annotation Syntax	211
In Function Signatures	211
Relationships	214
In Struct Definitions	215
Lifetime Elision	216
In Method Definitions.	218
The Static Lifetime	219
Generic Type Parameters, Trait Bounds, and Lifetimes.	219
Summary	220

11

WRITING AUTOMATED TESTS 221

How to Write Tests	222
Structuring Test Functions	222
Checking Results with the <code>assert!</code> Macro.	226
Testing Equality with the <code>assert_eq!</code> and <code>assert_ne!</code> Macros	229
Adding Custom Failure Messages	231
Checking for Panics with the <code>should_panic</code> Attribute	233
Using <code>Result<T, E></code> in Tests	236
Controlling How Tests Are Run	237
Running Tests in Parallel or Consecutively.	237
Showing Function Output.	238
Running a Subset of Tests by Name.	240
Ignoring Tests Unless Specifically Requested	241
Test Organization.	243
Unit Tests	243
Integration Tests	245
Summary	249

12

AN I/O PROJECT: BUILDING A COMMAND LINE PROGRAM 251

Accepting Command Line Arguments	252
Reading the Argument Values	252
Saving the Argument Values in Variables	254
Reading a File	255
Refactoring to Improve Modularity and Error Handling	256
Separating Concerns in Binary Projects	257
Fixing the Error Handling.	261
Extracting Logic from <code>main</code>	264
Splitting Code into a Library Crate	266
Adding Functionality with Test-Driven Development.	268
Writing a Failing Test.	268
Writing Code to Pass the Test	270

Working with Environment Variables	273
Writing a Failing Test for Case-Insensitive Search	273
Implementing the search_case_insensitive Function	274
Redirecting Errors to Standard Error	278
Checking Where Errors Are Written	278
Printing Errors to Standard Error	279
Summary	280

13

FUNCTIONAL LANGUAGE FEATURES: ITERATORS AND CLOSURES

281

Closures	282
Capturing the Environment	282
Inferring and Annotating Closure Types	284
Capturing References or Moving Ownership	286
Moving Captured Values Out of Closures	288
Processing a Series of Items with Iterators	292
The Iterator Trait and the next Method	293
Methods That Consume the Iterator	294
Methods That Produce Other Iterators	294
Closures That Capture Their Environment	295
Improving Our I/O Project	297
Removing a clone Using an Iterator	297
Clarifying Code with Iterator Adapters	300
Choosing Between Loops and Iterators	301
Performance in Loops vs. Iterators	301
Summary	302

14

MORE ABOUT CARGO AND CRATES.IO

303

Customizing Builds with Release Profiles	304
Publishing a Crate to Crates.io	305
Making Useful Documentation Comments	305
Exporting a Convenient Public API	308
Setting Up a Crates.io Account	311
Adding Metadata to a New Crate	312
Publishing to Crates.io	313
Publishing a New Version of an Existing Crate	314
Deprecating Versions from Crates.io	314
Cargo Workspaces	315
Creating a Workspace	315
Creating the Second Package in the Workspace	316
Installing Binaries with cargo install	320
Extending Cargo with Custom Commands	321
Summary	321

15

SMART POINTERS

323

Using Box<T> to Point to Data on the Heap	324
Storing Data on the Heap	325
Enabling Recursive Types with Boxes	325

Treating Smart Pointers Like Regular References	330
Following the Reference to the Value	330
Using Box<T> Like a Reference	331
Defining Our Own Smart Pointer.	331
Implementing the Deref Trait	332
Using Deref Coercion in Functions and Methods	333
Handling Deref Coercion with Mutable References	335
Running Code on Cleanup with the Drop Trait	335
A Reference-Counted Smart Pointer	339
Sharing Data	339
Cloning to Increase the Reference Count	341
The Interior Mutability Pattern	342
Enforcing Borrowing Rules at Runtime	343
Using Interior Mutability.	344
Allowing Multiple Owners of Mutable Data	350
Reference Cycles Can Leak Memory.	351
Creating a Reference Cycle	352
Preventing Reference Cycles Using Weak<T>	354
Summary	359

16

FEARLESS CONCURRENCY 361

Using Threads to Run Code Simultaneously	362
Creating a New Thread with spawn	363
Waiting for All Threads to Finish	364
Using move Closures with Threads	366
Transfer Data Between Threads with Message Passing	369
Transferring Ownership Through Channels.	372
Sending Multiple Values.	373
Creating Multiple Producers	374
Shared-State Concurrency.	375
Controlling Access with Mutexes.	375
Comparing RefCell<T>/Rc<T> and Mutex<T>/Arc<T>	381
Extensible Concurrency with Send and Sync	381
Transferring Ownership Between Threads.	382
Accessing from Multiple Threads.	382
Implementing Send and Sync Manually Is Unsafe	382
Summary	383

17

FUNDAMENTALS OF ASYNCHRONOUS PROGRAMMING 385

Parallelism and Concurrency	387
Futures and the Async Syntax	389
Our First Async Program	390
Defining the page_title Function	390
Executing an Async Function with a Runtime.	393
Concurrently Racing Two URLs Against Each Other	395
Applying Concurrency with Async	397
Creating a New Task with spawn_task.	397
Sending Data Between Two Tasks	400

Yielding Control to the Runtime	405
Building Our Own Async Abstractions	409
Streams: Futures in Sequence	411
A Closer Look at the Traits for Async	413
The Future Trait	413
The Pin Type and the Unpin Trait	415
The Stream Trait	421
Futures, Tasks, and Threads	423
Summary	425

18

OBJECT-ORIENTED PROGRAMMING FEATURES 427

Characteristics of Object-Oriented Languages	428
Objects Contain Data and Behavior	428
Encapsulation That Hides Implementation Details	428
Inheritance as a Type System and as Code Sharing	430
Using Trait Objects to Abstract Over Shared Behavior	431
Defining a Trait for Common Behavior	432
Implementing the Trait	433
Performing Dynamic Dispatch	436
Implementing an Object-Oriented Design Pattern	437
Attempting Traditional Object-Oriented Style	437
Encoding States and Behavior as Types	446
Summary	449

19

PATTERNS AND MATCHING 451

All the Places Patterns Can Be Used	452
match Arms	452
let Statements	453
Conditional if let Expressions	454
while let Conditional Loops	455
for Loops	456
Function Parameters	456
Refutability: Whether a Pattern Might Fail to Match	457
Pattern Syntax	459
Matching Literals	459
Matching Named Variables	459
Matching Multiple Patterns	460
Matching Ranges of Values	461
Destructuring to Break Apart Values	461
Ignoring Values in a Pattern	465
Adding Conditionals with Match Guards	470
Using @ Bindings	472
Summary	473

20

ADVANCED FEATURES

475

Unsafe Rust	476
Performing Unsafe Superpowers	476
Dereferencing a Raw Pointer	477
Calling an Unsafe Function or Method	479
Accessing or Modifying a Mutable Static Variable	484
Implementing an Unsafe Trait	485
Accessing Fields of a Union	486
Using Miri to Check Unsafe Code	486
Using Unsafe Code Correctly	488
Advanced Traits	488
Defining Traits with Associated Types	488
Using Default Generic Parameters and Operator Overloading	489
Disambiguating Between Identically Named Methods	492
Using Supertraits	495
Implementing External Traits with the Newtype Pattern	497
Advanced Types	498
Type Safety and Abstraction with the Newtype Pattern	498
Type Synonyms and Type Aliases	499
The Never Type That Never Returns	501
Dynamically Sized Types and the Sized Trait	502
Advanced Functions and Closures	504
Function Pointers	504
Returning Closures	506
Macros	508
The Difference Between Macros and Functions	508
Declarative Macros for General Metaprogramming	509
Procedural Macros for Generating Code from Attributes	511
Custom derive Macros	511
Attribute-Like Macros	517
Function-Like Macros	517
Summary	518

21

FINAL PROJECT: BUILDING A MULTITHREADED WEB SERVER

519

Building a Single-Threaded Web Server	520
Listening to the TCP Connection	521
Reading the Request	523
Looking More Closely at an HTTP Request	524
Writing a Response	525
Returning Real HTML	526
Validating the Request and Selectively Responding	528
Refactoring	530
From a Single-Threaded to a Multithreaded Server	531
Simulating a Slow Request	531
Improving Throughput with a Thread Pool	532
Graceful Shutdown and Cleanup	548
Implementing the Drop Trait on ThreadPool	548
Signaling to the Threads to Stop Listening for Jobs	550
Summary	553

A	
KEYWORDS	555
Keywords Currently in Use	555
Keywords Reserved for Future Use	557
Raw Identifiers	557
B	
OPERATORS AND SYMBOLS	559
Operators	559
Non-operator Symbols	562
C	
DERIVABLE TRAITS	567
Debug for Programmer Output	568
PartialEq and Eq for Equality Comparisons	568
PartialOrd and Ord for Ordering Comparisons	569
Clone and Copy for Duplicating Values	569
Hash for Mapping a Value to a Value of Fixed Size	570
Default for Default Values	570
D	
USEFUL DEVELOPMENT TOOLS	571
Automatic Formatting with rustfmt	571
Fix Your Code with rustfix	572
More Lints with Clippy	573
IDE Integration Using rust-analyzer	574
E	
EDITIONS	575
INDEX	577